CLASE 4 · PROGRAMACIÓN EN BASH

usuario@clase:~$ declare · function · trap

Variables, funciones y señales

Arrays, transformaciones, código reutilizable y scripts que no se dejan interrumpir

Basado en los capítulos 10 a 12 de *Bash Guide for Beginners*, de Machtelt Garrels

CAPÍTULO 10

Más sobre variables

Tipos y constantes, arrays, y operaciones sobre el contenido de una variable.

10

declare: dar un tipo a una variable

```
declare OPCIONES VARIABLE=valor
```

`-i`	Entero: lo asignado se evalúa como aritmética
`-a`	Array
`-r`	Solo lectura (constante)
`-x`	Exportada a los procesos hijos
`-p`	Muestra atributos y valor
`-f`	Solo nombres de funciones
`-t`	Atributo de rastreo (trace)

Con **+** en lugar de **-** se quita el atributo. Dentro de una función, declare crea variables locales.

Sin tipo: acepta cualquier cosa

```
$ VARIABLE=12
$ VARIABLE=string
$ echo $VARIABLE
string
```

Con tipo entero

```
$ declare -i VARIABLE=12
$ VARIABLE=string
$ echo $VARIABLE
0
$ declare -p VARIABLE
declare -i VARIABLE="0"
```

10.1.3 · CONSTANTES

readonly: variables que no cambian

```
$ readonly TUX=penguinpower
$ TUX=Mickeysoft
bash: TUX: readonly variable
$ unset TUX
bash: unset: TUX: cannot unset: readonly variable
```

```
readonly CONFIG="/etc/miapp.conf"
readonly MAX_INTENTOS=3
```

Una sola definición

Las constantes se declaran al principio del script; si hay que cambiar el valor, se cambia en un solo lugar.

Equivalente

declare -r hace lo mismo. **readonly -p** lista las de la sesión.

Para siempre

No se puede reasignar ni borrar con unset mientras dure la shell.

10.2.1 · CREAR ARRAYS

Arrays: muchos valores en una variable

Elemento por elemento

```
frutas[0]=manzana
frutas[1]=pera
```

Todo junto

```
frutas=(manzana p
era uva)
```

Declarado

```
declare -a frutas
```

Desde read

```
read -a frutas
```

Índices desde 0 · Sin tamaño máximo · Pueden tener huecos: [0], [5], [9] · Un atributo vale para todos los elementos

```
dias=([1]=lun [2]=mar [5]=vie)    # índices explícitos
```

Leer, contar y borrar elementos

```
$ A=(one two three)
$ echo ${A[2]}
three
$ echo $A[2]
one[2]
$ A[3]=four
$ echo ${A[@]}
one two three four
$ echo ${#A[@]}
4
$ unset 'A[1]'
$ echo ${A[@]}
one three four
```

`${A[2]}`	El elemento de índice 2
`${A[@]}`	Todos los elementos
`${#A[@]}`	Cantidad de elementos
`${!A[@]}`	Los índices usados
`unset 'A[1]'`	Borra un elemento
`unset A`	Borra el array

Las llaves son obligatorias: **$A[2]** es el primer elemento seguido del texto «[2]».
Para recorrerlo: **for x in "${A[@]}"**, con comillas.

10.2.4 · EJEMPLOS REALES

Dos usos prácticos de arrays

Distribuir una configuración

```
hosts=(web03 web04 web05 web06)
for h in "${hosts[@]}"; do
  scp "$conf" "$h:/usr/local/apache/conf"
  ssh "$h" sudo apachectl graceful
done
```

Agregar un servidor es agregar una palabra a la lista.

¿A quién le toca esta semana?

```
lista=("Ana <ana@ej.com>"
       "Luis <luis@ej.com>"
       "Sofía <sofia@ej.com>")
semana=$(date +%W)
semana=${semana#0}
i=$(( semana % ${#lista[@]} ))
echo "${lista[i]}"
```

Rotación semanal: el número de semana módulo la cantidad de personas.

10.3.2 · LONGITUD

${#VAR}: cuántos caracteres

```
$ echo $SHELL
/bin/bash
$ echo ${#SHELL}
9
$ A=(one two three)
$ echo ${#A[@]}
3
$ echo ${#A}
3
```

${#VAR}
Longitud del contenido de VAR.

${#A[@]}
Cantidad de elementos del array.

Cuidado con ${#A}
Es la longitud de **A[0]**. Aquí da 3 porque «one» tiene tres letras, no porque haya tres elementos.

10.3.3.1 · SUSTITUCIÓN

Valores por defecto y variables obligatorias

`${VAR:-PAL}`	Si VAR no existe o está vacía, usa PAL (VAR no cambia)
`${VAR:=PAL}`	Igual, pero además asigna PAL a VAR
`${VAR:?MSJ}`	Si no existe, muestra MSJ y termina el script
`${VAR:+PAL}`	Si VAR tiene valor, usa PAL; si no, nada

```
$ echo ${TEST:-test}
test
$ export TEST=a_string
$ echo ${TEST:-test}
a_string
$ [ -z "${COLUMNS:-}" ] && COLUMNS=80
```

```
#!/bin/bash
echo ${TESTVAR:?"Falta definir TESTVAR"}
echo "TESTVAR está definida, seguimos."
```

```
$ ./vartest.sh
./vartest.sh: line 2: TESTVAR: Falta definir TESTVAR
```

Recortar el principio o el final

`${VAR:INI:LON}`	LON caracteres desde la posición INI
`${VAR#PAT}`	Quita el prefijo más corto que coincide
`${VAR##PAT}`	Quita el prefijo más largo
`${VAR%PAT}`	Quita el sufijo más corto
`${VAR%%PAT}`	Quita el sufijo más largo

```
$ S=thisisaverylongname
$ echo ${S:6:5}
avery
$ echo ${S%name}
thisisaverylong
```

Con una ruta de archivo

```
$ f=/home/ana/informe.final.txt
$ echo ${f##*/}      # nombre
informe.final.txt
$ echo ${f%/*}       # directorio
/home/ana
$ echo ${f%.*}       # sin extensión
/home/ana/informe.final
$ echo ${f##*.}      # extensión
txt
```

Reemplazar dentro de una variable

`${VAR/PAT/TXT}`	Reemplaza la primera coincidencia
`${VAR//PAT/TXT}`	Reemplaza todas

```
$ S=thisisaverylongname
$ echo ${S/name/string}
thisisaverylongstring
$ f="mi archivo con espacios.txt"
$ echo ${f// /_}
mi_archivo_con_espacios.txt
```

Sobre todo un array

```
$ A=(one two one three one four)
$ echo ${A[@]#one}
two three four
$ echo ${A[@]##t*}
one one one four
```

Con [@] o [*], la operación se aplica a cada elemento.

CAPÍTULO 11

Funciones

Agrupar comandos bajo un nombre, pasarles argumentos y reutilizarlos.

11

11.1 · QUÉ SON

Una función: comandos con nombre

```
function saludar { echo "Hola, $1"; }
```

o, sin la palabra function:

```
saludar () {
  echo "Hola, $1"
}

saludar Ana      # se llama como un comando
```

Corren en la shell actual

No se crea un proceso nuevo, a diferencia de un script.

Errores típicos

Las llaves van separadas por espacios, y el cuerpo termina con ; o un salto de línea antes de la }.

Código de salida

El del último comando del cuerpo.

11.1.3 · PARÁMETROS

Una función tiene sus propios $1, $2...

```
#!/bin/bash
echo "Script: \$1 = $1"

mostrar () {
  echo "Función: \$1 = $1"
  echo "Me llamo $FUNCNAME, recibí $# argumentos"
}

mostrar otro valor
```

```
$ ./params.sh primero
Script: $1 = primero
Función: $1 = otro
Me llamo mostrar, recibí 2 argumentos
```

`$1 … $N`	Los argumentos de la función
`$#`	Se actualiza dentro de la función
`$0`	No cambia: sigue siendo el script
`FUNCNAME`	El nombre de la función

Al terminar, los parámetros vuelven a ser los del script.

11.1.3 · RETURN Y LOCAL

Devolver resultados y aislar variables

Un código: return

```
es_par () {
  (( $1 % 2 == 0 ))
}
if es_par 4; then echo "par"; fi
```

Un dato: echo y $( )

```
sumar () {
  echo $(( $1 + $2 ))
}
total=$(sumar 3 4)    # total=7
```

Variables locales: local

```
contar () {
  local n=0
  for f in "$1"/*; do (( n++ )); done
  echo "$n"
}
```

Sin **local**, n sería global y podría pisar una variable del script.

return N termina la función con el código N (0 a 255); queda en **$?**.

Funciones en ~/.bashrc

```
zless () {
  zcat "$@" | "$PAGER"
}

dir () {
  ls -lF --color=always "$@" | less -r
}
```

Definidas en **~/.bashrc**, quedan disponibles en cada terminal.

`type zless`	Dice qué es y muestra la definición
`declare -f`	Lista todas las funciones
`set`	También las muestra, junto a las variables
`unset -f dir`	Borra la función

Mejor que los alias

Aceptan argumentos en cualquier posición, condiciones y bucles.

Una biblioteca de funciones

funciones.sh

```
pathmunge () {
  if ! echo "$PATH" | grep -Eq "(^|:)$1($|:)"; then
    if [ "$2" = "after" ]; then
      PATH=$PATH:$1
    else
      PATH=$1:$PATH
    fi
  fi
}
```

En cada script

```
. ~/lib/funciones.sh
pathmunge /usr/local/sbin
pathmunge /opt/bin after
```

Se escribe una vez, se carga con **.** (o source) y se usa en todos los scripts.

Así funcionan los scripts de inicio, que cargan **/etc/rc.d/init.d/functions**.

11.2.3 · UN EJEMPLO COMPLETO

Respaldos con funciones parametrizadas

```
LOG=~/log/backup.log
log () { echo "$(date +%T) $*" >> "$LOG"; }

respaldar () {
  local dir=$1 nombre=$2; shift 2
  cd "$dir" || return 1
  tar cf "$nombre.tar" "$@"
  log "Comprimiendo $nombre.tar"
  bzip2 -f "$nombre.tar"
  scp "$nombre.tar.bz2" servidor:/respaldos/ &> /dev/null || return 2
  rm "$nombre.tar.bz2"; log "$nombre: listo"
}
```

```
respaldar ~/xml/db bash bash-programming/
```

Una función, muchos usos
Directorio, nombre y archivos llegan como argumentos.

Una función auxiliar
log escribe con la hora en un solo lugar.

Errores con código
return 1 o 2 según qué falló.

CAPÍTULO 12

Señales

Cómo se interrumpe un proceso, cómo terminarlo y cómo hacer que un script reaccione.

12

Las señales más comunes

Señal	Número	Significado
`SIGHUP`	1	Se cortó la terminal (hangup)
`SIGINT`	2	Interrupción desde el teclado (Ctrl+C)
`SIGKILL`	9	Terminar de inmediato: no se puede atrapar
`SIGTERM`	15	Pedido de terminación ordenada
`SIGSTOP`	varía	Detener el proceso: no se puede atrapar

Todas las señales: `kill -l` · documentación: `man 7 signal`

12.1.1 · SEÑALES Y LA SHELL

Señales desde el teclado

Ctrl+C

Envía **SIGINT** al trabajo en primer plano.

Ctrl+Z

Envía **SIGTSTP**: lo detiene y devuelve el control. Seguir con **fg** o **bg**.

Ctrl+Y

Suspensión diferida: se detiene al intentar leer de la terminal. No en todos los sistemas.

¿Por qué Ctrl+C no cierra la terminal?

Una shell interactiva sin trampas ignora SIGTERM y SIGQUIT, y atrapa SIGINT. SIGHUP sí la cierra y se reenvía a todos sus trabajos.

12.1.2 · KILL

kill: enviar una señal a un proceso

```
$ ps -ef | grep proceso_trabado
maud  5607  2214  0 20:05 pts/5  proceso_trabado
$ kill 5607          # SIGTERM
$ kill -9 5607       # SIGKILL, si no hubo caso
[1]+  Killed   proceso_trabado
```

`kill -s TERM 5607` `kill %1` `killall nombre`

`pkill nombre`

Primero lo amable

SIGTERM deja que el programa cierre archivos y limpie. Es la señal por defecto.

Último recurso

SIGKILL no da chance de limpiar: puede dejar archivos a medio escribir.

12.2 · TRAP

trap: reaccionar a una señal

```
trap 'COMANDOS' SEÑALES
```

```
#!/bin/bash
# traptest.sh
trap 'echo "¡Buu!"' SIGINT SIGTERM
echo "Mi PID es $$"
while :; do
  sleep 60
done
```

Ctrl+C y kill ya no lo detienen; hace falta **kill -9**.

`EXIT (o 0)`	Al terminar el script, por el motivo que sea
`ERR`	Cada vez que un comando falla
`DEBUG`	Después de cada comando
`trap '' INT`	Ignorar la señal
`trap - INT`	Volver al comportamiento normal

Limpiar al salir y proteger tareas críticas

Borrar el archivo de bloqueo

```
LOCK=/var/lock/miscript.lock
[ -f "$LOCK" ] && exit 0      # ya está corriendo
trap 'rm -f "$LOCK"' EXIT
touch "$LOCK"
tarea_larga
```

El bloqueo se borra siempre, aunque el script falle o lo interrumpan.

Una sección que no se interrumpe

```
trap 'echo "Espere, no se puede cancelar ahora"' I
NT
make install
trap - INT
```

Seguir una variable

```
declare -t VARIABLE=valor
trap 'echo "VARIABLE se usa aquí"' DEBUG
```